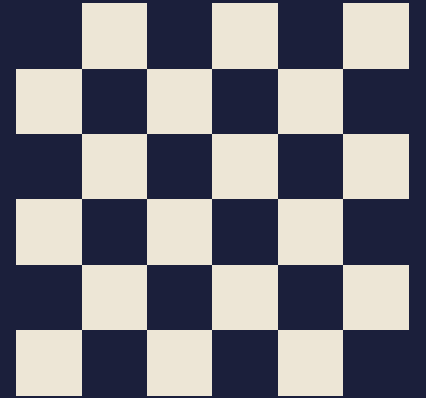


FUNDAMENTOS DE LA PROGRAMACIÓN

Bloque 1

Entorno, sintaxis y tipos básicos



Objetivos del bloque



Software y paradigmas

Reconocer los tipos básicos de software para el desarrollo de programas, así como los paradigmas de programación.



Entorno de desarrollo

Manejar correctamente las opciones básicas de un entorno de desarrollo, distinguiendo editor, compilador, intérprete e IDE.

Contenidos

- 1** 1.1 Entorno: instalación, entornos virtuales, depuración y git
- 2 1.2 Variables, identificadores y convenciones (PEP 8)
- 3** 1.3 Tipos de datos: int, float, str, bool
- 4 1.4 Operadores aritméticos, lógicos y de comparación
- 5** 1.5 Entrada / salida: input() y print()
- 6 1.6 Conversión de tipos (casting)
- 7** 1.7 Comentarios y documentación básica

Entrega 0 · Representar el tablero en consola (matriz 8×8)

Editor · Compilador · Intérprete · IDE

Editor

Escribe código como texto plano. No lo ejecuta ni lo entiende.

Compilador

Traduce todo el código a un ejecutable antes de correrlo (C, C++, Rust).

Intérprete

Lee y ejecuta línea a línea, sin generar un ejecutable. Así funciona Python.

IDE

Editor + ejecución + depuración + gestión de proyecto, todo en uno (VS Code).

Python es un lenguaje interpretado: no genera un ejecutable independiente.

Compilado vs. interpretado

COMPILADO (C, C++, Rust...)

Código fuente (archivo .c)



Compilador traduce TODO de una vez



Se genera un ejecutable → se ejecuta

INTERPRETADO (Python)

Código fuente (archivo .py)



El intérprete (CPython) lee y traduce



Ejecuta y muestra el resultado, línea a línea

Por eso Python no genera un "programa.exe": cada vez que ejecutas `python archivo.py`, el intérprete vuelve a leer el archivo entero desde el principio.

Paradigmas de programación



Imperativo / procedural

Secuencia de instrucciones que modifican el estado paso a paso. Punto de partida del curso.



Orientado a objetos

El programa se organiza en objetos que combinan datos y comportamiento (clases).



Funcional

Funciones puras, sin modificar estado. Python ofrece map, filter, lambda.



Declarativo

Se describe QUÉ se quiere obtener, no CÓMO (ej. SQL).

Instalar Python, paso a paso

1

Entra en python.org/downloads

Es la web oficial. Descarga siempre desde ahí, nunca de un enlace de terceros.

2

Descarga el instalador de Python 3

Elige la versión para tu sistema operativo (Windows, macOS, Linux).

3

Ejecuta el instalador

Sigue los pasos del asistente de instalación.

4

Verifica la instalación

Abre una terminal y escribe `python --version` (o `python3 --version`).

En Windows, en el paso 3, marca la casilla "Add Python to PATH". Si no la marcas, la terminal no reconocerá el comando `python` y habrá que reinstalar.

Instalar VS Code y extensiones

1

Descarga VS Code

Desde code.visualstudio.com, para tu sistema operativo.

2

Instala la extensión "Python" (Microsoft)

Icono de piezas de puzle en la barra lateral izquierda → buscar "Python" → Install.

3

Instala la extensión "Jupyter" (Microsoft)

Mismo proceso: permite trabajar con notebooks (.ipynb) dentro de VS Code.

4

Abre la carpeta de tu proyecto

Archivo → Abrir carpeta... Ahí guardarás todos los .py de las entregas.

Un entorno virtual por programa

Un entorno virtual (venv) es una copia aislada de Python con sus propias librerías, independiente de las demás. Cada proyecto tiene la suya.

SIN entorno virtual

Todos los programas comparten las mismas librerías instaladas "a nivel global". Un proyecto que necesita la versión 1 de una librería choca con otro que necesita la versión 2.

CON entorno virtual

Cada carpeta de proyecto tiene su propia "caja" de librerías, sin interferir con las demás ni con el resto del sistema.

Regla del bloque

Cada programa nuevo, en su propia carpeta, con su propio entorno virtual. No se reutiliza el entorno de una entrega anterior ni se instala nada "a nivel global": así se evita que un problema de otro proyecto afecte a este.

Crear y activar tu entorno virtual

1

Abre la terminal en la carpeta del proyecto

La misma terminal integrada de VS Code (Ctrl + `).

2

Créalo: `python -m venv venv`

Crea una carpeta "venv" con una copia de Python, dentro de tu proyecto.

3

Actívalo

Windows: `venv\Scripts\activate` · macOS/Linux: `source venv/bin/activate`

4

Selecciona el intérprete en VS Code

Abajo a la derecha, elige el Python de `.../venv/...` — así VS Code usa ese entorno.

Sabrás que está activo porque el nombre (venv) aparece al principio de la línea de la terminal.

Tres formas de ejecutar Python

Dónde	Cómo se ejecuta	Cuándo usarlo
REPL (terminal)	Línea a línea, al pulsar Enter. No se guarda nada.	Probar algo rápido, sin dejar rastro.
Script (.py)	Se ejecuta el archivo entero, de arriba abajo, de una vez.	Entregas y proyectos — nuestro caso todo el curso.
Notebook (.ipynb)	Por celdas: cada celda se ejecuta por separado y el resultado queda debajo.	Explorar datos, prototipar, seguir la clase.

Tu primer programa

```
print("Hola, mundo")
```

1

Activa el entorno virtual del proyecto

Si no lo has hecho ya: `source venv/bin/activate` (o `venv\Scripts\activate` en Windows).

2

Guarda el archivo como `hola_mundo.py`

En la carpeta de tu proyecto (extensión `.py`, no `.txt`).

3

Abre la terminal integrada

Menú Terminal → Nueva terminal, o el atajo `Ctrl + `` (acento grave).

4

Ejecuta: `python hola_mundo.py`

Debe aparecer "Hola, mundo" en la terminal.

El botón ▷ "Run" de VS Code hace exactamente lo mismo por detrás: ejecuta `python hola_mundo.py` por ti.

Cuando algo falla: errores de entorno



"python" no se reconoce

El PATH no está bien configurado. Reinstala Python marcando "Add Python to PATH".



Import "..." could not be resolved

VS Code no tiene seleccionado el intérprete de Python. Selecciónalo abajo a la derecha.



SyntaxError / IndentationError

Hay un error de escritura. Lee la línea exacta que señala el mensaje de error.

Leer el mensaje de error con calma es la habilidad más útil de todo el curso: casi siempre dice, literalmente, qué ha pasado y en qué línea.

Depurar en tiempo de ejecución

Depurar (debug) es pausar el programa mientras se ejecuta para ver, en cada instante, qué valor tiene cada variable — en vez de adivinarlo a base de `print()`.

Con `print()`

Añadir y quitar `print(variable)` por todo el código, ejecutar una y otra vez, y borrar los prints al terminar. Lento y deja rastro si se te olvida alguno.

Con el depurador

El programa se pausa solo donde tú decidas (un breakpoint), sin tocar ni una línea de código, y ves TODAS las variables a la vez.

VS Code trae un depurador integrado para Python, listo para usar sin instalar nada más.

El depurador de VS Code, paso a paso

```
TAMANO_TABLERO: int = 8  
fila: int = 4  
centro: int = TAMANO_TABLERO // 2 ← breakpoint aquí  
print(centro)
```

1

Pon un breakpoint

Clic en el margen izquierdo, junto al número de línea: aparece un punto rojo.

2

Inicia la depuración con F5

O el icono ▶ con un bicho, en el panel "Run and Debug". El programa se ejecuta hasta el breakpoint y se detiene.

3

Inspecciona las variables

En el panel lateral "Variables" ves el valor exacto de cada una en ese instante.

4

Avanza con Step Over (F10) o Continue (F5)

Step Over ejecuta una línea más; Continue sigue hasta el siguiente breakpoint (o hasta el final).

¿Qué es git?

Git es un sistema de control de versiones: guarda el historial completo de los cambios de tu código, como un "histórico de guardado" muy potente.

1

Deshacer con seguridad

Vuelve a cualquier versión anterior de tu proyecto si algo se rompe.

2

Historial de cambios

Cada commit guarda qué cambió, cuándo y por qué (el mensaje del commit).

3

Copia de seguridad

Subido a GitHub, tu código sobrevive aunque se estropee tu ordenador.

4

Colaborar sin pisarse

Varias personas pueden trabajar en el mismo proyecto sin sobrescribirse el trabajo.

GitHub es una web que aloja repositorios git en la nube — git es la herramienta, GitHub es donde guardas la copia remota.

Cómo funciona git, en tres pasos



`git add` marca qué cambios entran en el próximo commit · `git commit` guarda esos cambios en el historial local, con un mensaje · `git push` sube ese historial al repositorio remoto (GitHub).

Tu primer commit y subida a GitHub

1

Crea un repositorio en GitHub

Cuenta en github.com → New repository → nómbralo y créalo (sin plantillas).

2

Inicializa git en tu carpeta

`git init` — convierte la carpeta del proyecto en un repositorio git.

3

Prepara y confirma tus cambios

`git add .` seguido de `git commit -m "Primera entrega: tablero en consola"`.

4

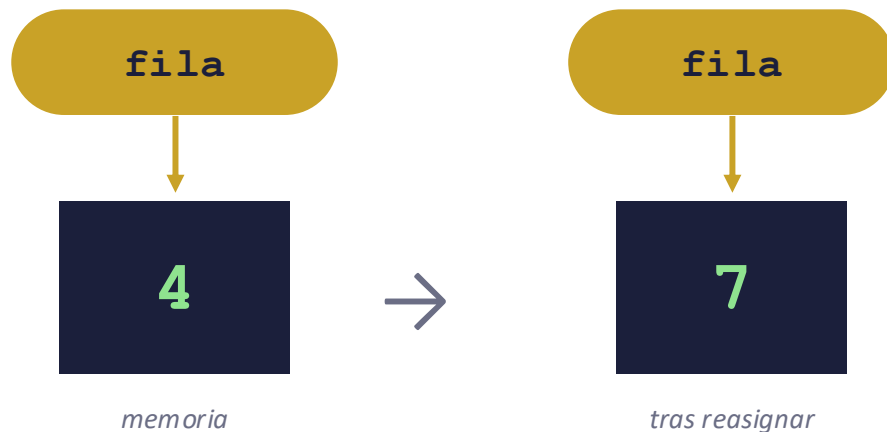
Conecta con GitHub y sube el historial

`git remote add origin <url-del-repositorio>` y luego `git push -u origin main`.

El mensaje del commit importa: "cambios" no dice nada; "Añade la Entrega 0: tablero en consola" sí.

¿Qué es una variable?

Una **variable** es un **nombre** que apunta a un valor guardado en memoria. Se crea la primera vez que le asignas algo con `=`.



```
fila: int = 4
print(fila)    # 4
fila: int = 7
print(fila)    # 7
```

El mismo nombre puede apuntar a otro valor más tarde (reasignación): `fila: int = 4` y luego `fila: int = 7` no crean una variable nueva, cambian a qué apunta `fila`. Repetir el tipo no es obligatorio para que Python funcione, pero en este curso se anota siempre, también al reasignar. Python no reserva un tamaño fijo de memoria como C: el nombre simplemente apunta a donde esté el valor.

Declarar variables: ejemplo guiado

```
nombre_jugador: str = "Blancas"  
turno: int = 1  
en_jaque: bool = False  
  
print(nombre_jugador, turno, en_jaque)
```

Salida por consola

```
Blancas 1 False
```

print() separa los argumentos con un espacio por defecto, y los convierte a texto automáticamente (aunque en_jaque sea bool, no str).

Regla del bloque, muy importante

Toda variable que el programa vaya a usar se declara, inicializa y anota con su tipo antes de utilizarse. Nada de variables "que ya aparecerán más adelante": si tu programa necesita una fila, un ancho o un carácter, esa variable existe desde el principio, con un valor real y su tipo escrito.

Identificadores válidos e inválidos

	<code>2casillas</code>	empieza por un dígito
	<code>casilla_2</code>	letras, dígitos y guion bajo, no empieza por dígito
	<code>_temp</code>	válido, aunque inusual fuera de convenciones internas
	<code>class</code>	es una palabra reservada de Python
	<code>Fila</code>	válido, pero distinto de fila (sensible a mayúsculas)
	<code>fila-actual</code>	el guion se interpreta como una resta
	<code>tablero8x8</code>	los dígitos y letras se pueden mezclar

Convenciones de estilo — PEP 8

PEP 8 es la guía de estilo oficial de Python. No es obligatoria para que el código funcione, pero sí para que sea legible — y se evaluará en las entregas.

snake_case

para variables y funciones

```
fila_actual
```

UPPER_CASE

para constantes

```
TAMANO_TABLERO
```

CamelCase

reservado para clases

```
TableroAjedrez
```

Nombres descriptivos

evita una sola letra

```
fila, no f
```

Espacios

alrededor de = y :

```
x: int = 1
```

79 caracteres

máximo recomendado por línea

```
—
```

VS Code (Pylance) subraya automáticamente muchas violaciones de PEP 8. No ignorar esos avisos: la ausencia de warnings puntúa en la evaluación de las entregas.

Anotaciones de tipo (obligatorias)

`nombre_variable: tipo = valor_inicial`

```
fila: int = 4
casilla_clara: str = "."
eval_posicion: float = 3.5
```

```
fila: int = 4
fila = "cuatro"
# Pylance avisa: se esperaba
# int, se asignó str
```

Elige el tipo correcto, no el que "funcione"

Python 3 no distingue int de long (a diferencia de C/Java): un único int que crece solo, uses el rango que uses.

No uses float si el valor siempre es entero · **no uses str** para guardar un número · **no uses int** a modo de bool disfrazado

Tipos de datos básicos

int

```
fila: int = 4
```

Números enteros. Filas y columnas del tablero: 0 a 7.

float

```
eval: float = 3.5
```

Números con decimales (con punto, no coma).

str

```
casilla: str = "e4"
```

Texto entre comillas. Indexable: casilla[0] → 'e'.

bool

```
jaque: bool = True
```

Solo True o False. Subclase de int (1 / 0).

Tamaño y precisión de los tipos

Tipo	Tamaño	Rango / precisión
<code>bool</code>	1 o 0 por dentro	Solo True o False
<code>int</code>	Crece con el número	Sin límite superior fijo (precisión arbitraria)
<code>float</code>	Siempre 64 bits	IEEE-754 doble precisión, ~15-17 dígitos
<code>str</code>	Crece con la longitud	Se consulta con <code>len()</code>

```
import sys
sys.getsizeof(True)      # unos 28 bytes
sys.getsizeof(3.14)      # 24 bytes, siempre igual
sys.getsizeof("hola")    # crece con la longitud
```

Los valores exactos varían según la versión de Python y el sistema operativo — la idea importa más que memorizar el número.

Cadenas de texto: indexar y trocear

```
casilla: str = "e4"
```



*índices positivos arriba, negativos (desde el final)
abajo*

```
casilla[0]      # 'e'  
casilla[1]      # '4'  
casilla[-1]     # '4' (último)  
len(casilla)    # 2
```

Las cadenas son inmutables: no se puede hacer `casilla[0] = "E"` para modificarla. Hay que crear una cadena nueva. Y si la casilla tuviera dos dígitos ("e10"), `casilla[1]` seguiría siendo el segundo carácter, no "el número de columna completo".

¿Qué tipo es esto? (antes de ejecutar)

`type(5)` → `int`

`type(5.0)` → `float`

`type("5")` → `str`

`type(True)` → `bool`

`type(5 == 5.0)` → `bool` (**¡el valor es True!**)

`type("True")` → `str` (**¡ojo, no bool!**)

5 == 5.0 compara el VALOR, no el tipo (da True); "True" con comillas es una cadena de 4 caracteres, no el booleano.

Ahora te toca a ti

Ficha de jugador (*≈ 5 minutos*)

Declara, inicializa y anota cuatro variables: `nombre_jugador` (str), `color_piezas` (str), `turno` (int) y `en_jaque` (bool). Imprímelas todas con una sola llamada a `print()`.

```
nombre_jugador: str = ???  
# completa tú el resto...
```

Pista

Un solo `print()` puede recibir varias variables separadas por comas: `print(nombre_jugador, color_piezas, turno, en_jaque)`.

Cuando termines, compara tu solución con la de la persona de al lado: ¿elegisteis los mismos tipos?

Operadores aritméticos

Operador	Significado	Ejemplo	Resultado
+	Suma	5 + 3	8
-	Resta	5 - 3	2
*	Multiplicación	5 * 3	15
/	División (siempre float)	7 / 2	3.5
//	División entera	7 // 2	3
%	Módulo (resto)	7 % 2	1
**	Potencia	2 ** 3	8

El operador % es especialmente útil para tableros: fila % 2 permite saber si una fila es par o impar.

La trampa: división entera con negativos

```
7 // 2      # 3
-7 // 2     # -4 ;no -3!
7 % 2       # 1
-7 % 2      # 1 ;no -1!
```

¿-7 // 2 da -3 o -4?

Calcúlalo a mano antes de mirar el código. La intuición de "redondear hacia cero" (como en una calculadora) falla aquí.

Python redondea hacia menos infinito, no hacia cero

-7 // 2 da -4 (no -3): es distinto de C o Java, que redondean hacia cero.

-7 % 2 da 1 (no -1), porque Python garantiza siempre que $(a // b) * b + (a \% b) == a$. Es una garantía matemática del lenguaje, no un capricho.

Comparar cosas que no son del mismo tipo

```
1 == 1.0
```

True*compara el valor, no el tipo*

```
"10" == 10
```

False*un str nunca es igual a un int*

```
"10" == "10"
```

True*mismo tipo, mismo valor*

```
0 < 5 < 10
```

True*comparación encadenada: (0<5) and (5<10)*

"10" == 10 da False: aunque "parezcan lo mismo", un texto nunca es igual a un número en Python. Es un error clásico al leer datos con `input()` y compararlos sin convertir.

Operadores lógicos: una condición real

and (ambas verdaderas) · or (al menos una verdadera) · not (niega el valor)

```
fila: int = 7
columna: int = 8
dentro_del_tablero: bool = (fila >= 0) and (fila <= 7) and (columna >= 0) and (columna <= 7)
```

¿True o False, con columna = 8?

Da False. Las columnas válidas son 0 a 7 (8 valores en total, empezando en 0): columna = 8 queda fuera del tablero, aunque "8" suene como un valor razonable para un tablero de 8 columnas.

Este es el error de "índice fuera de rango" más típico al empezar: contar desde 1 en vez de desde 0. Tenlo siempre presente cuando trabajes con posiciones.

print(): salida por consola

```
print("Turno de", "blancas")  
# Turno de blancas    (argumentos separados por espacio)  
  
print("a", "b", "c", sep="-")  
# a-b-c    (sep cambia el separador)  
  
print("Cargando", end="...")  
# Cargando...    (end cambia qué va al final, sin salto de línea)  
  
fila: int = 4  
print(f"Fila seleccionada: {fila}")  
# Fila seleccionada: 4
```

Las f-strings (f"...{variable}...") son la forma recomendada de insertar variables en un texto: más legibles que concatenar con +.

input(): entrada por teclado

Detiene la ejecución hasta que el usuario escribe algo y pulsa Enter.

```
nombre: str = input("¿Cómo te llamas? ")  
  
print(type(nombre))  
# <class 'str'> ← siempre, aunque escribas un número
```

input() SIEMPRE devuelve texto (str)

Aunque el usuario escriba "7", input() devuelve la cadena "7", no el número 7. Si necesitas operar con ese valor (sumarlo, compararlo como número...), tienes que convertirlo explícitamente con int() o float() — es el casting del apartado 1.6, y es uno de los pasos que MÁS se olvida al empezar.

Construir una fila del tablero, a mano

Sin bucles todavía: así se construiría, casilla a casilla, una fila usando varios `print()` con `end`:

```
print(".", end="")  
print("#", end="")  
print(".", end="")  
print("#", end="")  
print(".", end="")  
print("#", end="")  
print() # salto de línea final
```

Salida:

. # . # . #

Cada `print(..., end="")` evita el salto de línea automático, así el siguiente carácter se pega al anterior. El último `print()` vacío sí salta de línea.

Este ejemplo es deliberadamente tedioso: es la motivación perfecta para el Bloque 2 ("esto se puede hacer con un for en una sola línea").

Conversión de tipos (casting)

`int()` · `float()` · `str()` · `bool()` convierten un valor de un tipo a otro, de forma explícita.

```
fila_texto: str = "4"
```

`int()`

```
fila: int = 4
```

```
float("3.5")
```

```
str(8)
```

⚠ `int("ocho")` → *ValueError: invalid literal for int() with base 10: 'ocho'*

Imprescindible tras un `input()`, que siempre devuelve `str` y no se puede sumar ni comparar como número sin convertir antes.

Leer un error de casting

Antes de ejecutar: ¿qué crees que pasará?

```
edad: int = int("veinte")
```

```
Traceback (most recent call last):  
  File "programa.py", line 1, in <module>  
    edad: int = int("veinte")  
ValueError: invalid literal for int() with base 10: 'veinte'
```

Truco para leer cualquier traceback: empieza por la última línea (tipo de error + mensaje) y sube solo si necesitas más contexto.

Castings encadenados: cuidado al truncar

```
int(float("3.9"))    # 3    (trunca, no redondea)
int("3.9")           # ValueError (int() no admite un punto decimal)
```

¿Por qué falla `int("3.9")` pero `int(float("3.9"))` no?

int() puede convertir un str a int SOLO si el texto representa un entero exacto ("3", "-8"...). Si tiene un punto decimal, falla, aunque "parezca" un número.

float() sí entiende decimales: `float("3.9")` da 3.9. Y **int()** Sí sabe convertir un float a int — pero **trunca la parte decimal, no redondea**: `int(3.9)` da 3, no 4.

El bug clásico: sumar como si fuera texto

```
a: str = input("Primer número: ")  
b: str = input("Segundo número: ")  
print(a + b)  # con a='5' y b='3', imprime '53', ¡no 8!
```

¿Por qué pasa esto?

input() devuelve str. El operador + sobre dos str no suma números: concatena texto. "5" + "3" pega las dos cadenas y da "53".

```
print(int(a) + int(b))  # 8 -- ahora sí, con casting
```

Ahora te toca a ti

Calculadora rápida (*≈ 7 minutos*)

Pide dos números por teclado. Conviértelos a `int`. Muestra, con f-strings: su suma, su resta, y si el primero es mayor que el segundo (usa un operador de comparación).

```
a_texto: str = input("Primer número: ")  
# convierte, calcula, e imprime con f-strings...
```

Pista

`input()` siempre devuelve `str`: conviértelos con `int()` antes de sumar, restar o comparar — si no, tendrás el bug de la diapositiva anterior.

Reto extra si te sobra tiempo: ¿qué pasa si divides los dos números con `//` y con `%`?

Comentarios y documentación

```
"""  
Entrega 0 - Fundamentos de la Programación.  
Representa un tablero de ajedrez 8x8 en la consola.  
"""  
  
TAMANO_TABLERO: int = 8 # constante: filas y columnas
```

- Comentar el porqué, no repetir lo que el código ya dice
- Mantener los comentarios actualizados: si contradicen al código, son peores que no tener ninguno
- Docstrings para documentar el propósito de archivos y funciones

Antes y después: por qué comentar bien

SIN CONTEXTO — ¿para qué sirve esto?

```
t: int = 8  
c: int = t // 2  
print(c)
```

CON NOMBRES Y COMENTARIO CLAROS

```
TAMANO_TABLERO: int = 8  
centro: int = TAMANO_TABLERO // 2 # centro  
print(centro)
```

El objetivo no es "comentar más"

sino comentar lo que el nombre de la variable no deja claro por sí solo. Buenos nombres (TAMANO_TABLERO, centro) reducen la necesidad de comentarios; no la eliminan del todo.

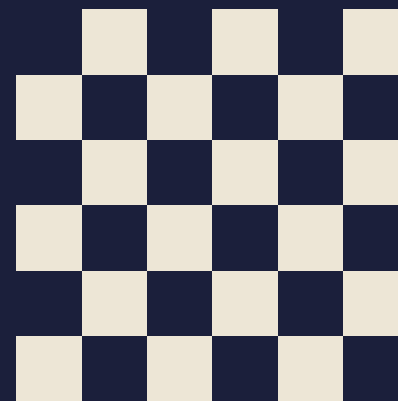
Mal comentario: `i = i + 1 # incrementamos i` (repite lo obvio).

Buen comentario: explica el porqué de una decisión que el código no deja ver.

Resumen del Bloque 1

- ✓ Distinguir editor / compilador / intérprete / IDE
- ✓ Instalar y configurar Python y VS Code
- ✓ Ejecutar código como REPL, script y notebook
- ✓ Declarar variables con PEP 8 y tipo anotado
- ✓ Usar operadores aritméticos, de comparación y lógicos
- ✓ Convertir tipos explícitamente (casting)
- ✓ Guardar y subir código con git y GitHub
- ✓ Reconocer los paradigmas de programación
- ✓ Crear y activar un entorno virtual por proyecto
- ✓ Depurar en tiempo de ejecución con breakpoints
- ✓ Elegir el tipo correcto: int, float, str, bool
- ✓ Leer y escribir por consola: input() y print()
- ✓ Documentar el código con comentarios y docstrings

Todo esto — y solo esto — es lo necesario para resolver la Entrega 0: aún no hay bucles ni condicionales, así que el reto está en representar el tablero 8×8 usando exactamente estas herramientas.



Próximo bloque

Bloque 2 - Control de flujo

Condicionales y bucles: la Entrega 0 se ampliará usando for para generar el tablero de forma dinámica.